



at in Linux

1. What is the **at** Command?

The **at** command is used in Linux to **schedule a command or script to run once at a specific time in the future**.

Unlike **cron**, which runs jobs repeatedly, **at** **executes a job only one time**.

Example use cases:

- Restart a service after 30 minutes
- Run a script tonight at 10 PM
- Schedule a system shutdown
- Execute a backup later in the day

2. Basic Syntax

at TIME

After running the command, you will enter **interactive mode** where you type the command to execute.

End the input with:

CTRL + D

Example:

at 15:00

Then type:

```
echo "Hello World" > /tmp/test.txt
```

Press:

CTRL + D

The command will run at **3 PM**.

3. How **at** Works

The **at** command works using a background service called:

atd

This daemon monitors scheduled jobs and executes them at the specified time.

4. Check if **atd** Service is Running

```
systemctl status atd
```

Start service if needed:

```
sudo systemctl start atd
```

Enable at boot:

```
sudo systemctl enable atd
```

5. Common Time Formats

`at` accepts many **time formats**.

Format	Example
Specific time	<code>at 15:30</code>
AM/PM format	<code>at 3:30 PM</code>
After minutes	<code>at now + 10 minutes</code>
After hours	<code>at now + 2 hours</code>
Tomorrow	<code>at 10:00 tomorrow</code>
Specific date	<code>at 10:00 Mar 20</code>
Next week	<code>at now + 1 week</code>

6. Example Commands

Example 1: Run Command at Specific Time

`at 18:00`

Command:

```
touch /tmp/testfile
```

Meaning:

Run command at **6 PM**.

Example 2: Run After 5 Minutes

at now + 5 minutes

Command:

```
echo "Test job executed" >> /tmp/test.log
```

Example 3: Run a Script

at 22:00

Command:

```
/home/user/backup.sh
```

Meaning:

Run the script at **10 PM**.

Example 4: Using Pipe with **at**

Instead of interactive mode:

```
echo "date > /tmp/current_time.txt" | at now + 1 minute
```

Meaning:

After **1 minute**, the date will be saved.

7. Listing Scheduled Jobs

To see all scheduled jobs:

atq

Example output:

2 Mon Mar 16 15:30:00 2026 a root

Explanation:

Field	Meaning
2	Job number
Date/time	Scheduled execution time
a	Queue name
root	User

8. View Job Details

To see the command inside a job:

`at -c job_number`

Example:

`at -c 2`

This shows the full script stored by the `at` command.

9. Remove Scheduled Job

Delete a job:

`atrm job_number`

Example:

`atrm 2`

10. Location of `at` Jobs

Scheduled jobs are stored in:

`/var/spool/at/`

11. Access Control

Two files control who can use `at`.

Allow file

/etc/at.allow

Users listed here **can use at**.

Deny file

/etc/at.deny

Users listed here **cannot use at**.

Rules:

1. If **at.allow** exists → only listed users allowed.
 2. If **at.allow** doesn't exist → check **at.deny**.
 3. If neither exists → all users allowed.
-

12. Advantages of at

- Easy to schedule **one-time tasks**
 - Flexible time formats
 - Useful for **temporary scheduling**
-

13. Limitations

- Runs only **once**
- Not suitable for repetitive tasks
- Requires **atd service running**



Crontab in Linux

cron is a **time-based job scheduler** in Linux.

It allows you to automatically run commands or scripts at specific times.

Used for:

- Backups
 - Log cleanup
 - Monitoring scripts
 - System maintenance
 - Report generation
-

1 What is cron?

cron is a background service (daemon) that executes scheduled tasks.

The configuration file used to schedule jobs is called **crontab** (cron table).

2 Cron Service

In modern systems (using systemd):

Start cron:

```
systemctl start crond
```

Enable at boot:


```
systemctl enable crond
```

Check status:

```
systemctl status crond
```

On Debian/Ubuntu:

```
systemctl status cron
```

3 Crontab Command

- ◆ **Edit crontab**

```
crontab -e
```

- ◆ **List crontab**

```
crontab -l
```

- ◆ **Remove crontab**

```
crontab -r
```

- ◆ **Edit another user's crontab (root)**

```
crontab -u username -e
```

4 Crontab Format

Each line has **5 time fields + command**

```
* * * * * command
| | | | |
| | | | └─ Day of Week (0-7)
| | | └─── Month (1-12)
| | └───── Day of Month (1-31)
| └──────── Hour (0-23)
└────────── Minute (0-59)
```

5 Time Field Explanation

Field	Allowed Values
Minute	0–59
Hour	0–23
Day of Month	1–31
Month	1–12
Day of Week	0–7 (0 or 7 = Sunday)

6 Special Symbols in Crontab

Symbol	Meaning
*	Every value
,	Multiple values
–	Range
/	Step value

7 Basic Examples

◆ Run every minute

```
* * * * * /home/user/script.sh
```

◆ Run every day at 2 AM

```
0 2 * * * /home/user/backup.sh
```

◆ Run every Sunday at 5 PM

```
0 17 * * 0 /home/user/weekly.sh
```

◆ Run every 10 minutes

```
*/10 * * * * /home/user/check.sh
```

◆ Run every Monday–Friday at 9 AM

```
0 9 * * 1-5 /home/user/work.sh
```

◆ Run on 1st of every month

```
0 0 1 * * /home/user/monthly.sh
```

8 Redirecting Output (Very Important 🔥)

By default, cron emails output to user.

Better practice:

```
0 2 * * * /home/user/script.sh > /home/user/output.log 2>&1
```

Explanation:

- `>` → redirect output
- `2>&1` → redirect errors

To discard output:

```
0 2 * * * /home/user/script.sh > /dev/null 2>&1
```

Cron Logs

On RHEL/CentOS:

```
cat /var/log/cron
```

On Ubuntu:

```
grep CRON /var/log/syslog
```

System-Wide Cron Jobs

 **/etc/crontab**

System-wide cron file.

Format includes extra field:

```
* * * * * user command
```

Example:

```
0 2 * * * root /usr/local/bin/backup.sh
```

Special Directories

Directory	Purpose
<code>/etc/cron.hourly/</code>	Hourly jobs
<code>/etc/cron.daily/</code>	Daily jobs
<code>/etc/cron.weekly/</code>	Weekly jobs
<code>/etc/cron.monthly/</code>	Monthly jobs

Place script in these directories.

Make executable:

```
chmod +x script.sh
```

Environment in Cron

Cron runs in limited environment.

Always use:

- Full path of command
- Absolute path for scripts

Instead of:

```
backup.sh
```

Use:

```
/home/user/backup.sh
```

13 Check Cron Permissions

Two files control access:

```
/etc/cron.allow
```

```
/etc/cron.deny
```

If cron.allow exists:

- Only listed users can use cron.
-

14 Practical Scenarios

◆ Automatic Backup Every Day

```
0 1 * * * tar -cvzf /backup/home_$(date +%F).tar.gz /home
```

Note:

Escape % in cron using \%.

◆ Delete Files Older Than 7 Days

```
0 3 * * * find /tmp -type f -mtime +7 -delete
```

◆ Restart Service Daily

```
0 4 * * * systemctl restart httpd
```

15 Difference: Cron vs at

Cron	at
Repeated jobs	One-time job
Time-based schedule	Run once at specific time

Example `at`:

`at 5pm`

16 Common Errors

✗ Cron not running

Check:

`systemctl status crond`

✗ Script not executing

Check:

- Execute permission
- Absolute path
- Environment variables
- Log files

17 Special Strings (Shortcuts)

Instead of numeric values:

Shortcut	Meaning
@reboot	Run at startup
@daily	Once per day
@weekly	Once per week
@monthly	Once per month
@yearly	Once per year

Example:

```
@reboot /home/user/startup.sh
```

18 Real-time Scenarios

1. Database Backup (Very Common)

Companies schedule automatic **database backups every night**.

Scenario

Backup MySQL database daily at **2 AM**.

Cron job:

```
0 2 * * * /scripts/db_backup.sh >> /var/log/db_backup.log 2>&1
```

Example script (**db_backup.sh**):

```
mysqldump -u root -pPassword mydb > /backup/mydb_$(date +%F).sql
```

Purpose:

- Prevent data loss
 - Automatic recovery in case of failure
-

2. Log File Cleanup

Servers generate huge logs. Old logs must be deleted automatically.

Scenario

Delete logs older than **7 days**.

Cron job:

```
0 3 * * * find /var/log/myapp/ -type f -mtime +7 -delete
```

Meaning:

- Run daily at **3 AM**
 - Remove logs older than **7 days**
-

3. Monitoring Disk Usage

System admins monitor disk usage automatically.

Scenario

Check disk usage every **10 minutes**.

Cron job:

```
*/10 * * * * /scripts/disk_check.sh
```

Example script:

```
df -h | grep "/dev/sda1" | awk '{print $5}' | sed 's/%//' | while read output;
do
if [ $output -gt 80 ]; then
echo "Disk usage is above 80%" | mail -s "Disk Alert" admin@example.com
fi
done
```

Purpose:

- Prevent disk full issues
-

4. Restarting Services Automatically

Some services may need periodic restart.

Scenario

Restart **nginx** every day at **4 AM**.

Cron job:

```
0 4 * * * systemctl restart nginx
```

Purpose:

- Clear memory leaks

- Improve stability
-

5. Temporary File Cleanup

Applications create temporary files.

Scenario

Delete files older than **3 days** in `/tmp`.

Cron job:

```
0 1 * * * find /tmp -type f -mtime +3 -delete
```

Runs daily at **1 AM**.

6. Automated File Transfer

Cron can automate file transfers between servers.

Scenario

Transfer files to backup server every night.

Cron job:

```
30 2 * * * rsync -av /data backup@192.168.1.10:/backup/
```

Purpose:

- Disaster recovery
 - Data replication
-

7. Security Updates

Automate package updates.

Scenario

Update system packages every Sunday.

Cron job:

```
0 3 * * 0 yum update -y
```

Purpose:

- Security patching
 - System maintenance
-

8. Generate Reports

Applications generate daily reports.

Scenario

Generate sales report at **midnight**.

Cron job:

```
0 0 * * * /scripts/sales_report.sh
```

Output may be emailed to management.

9. Website Health Monitoring

Cron can monitor website availability.

Scenario

Check website every **5 minutes**.

Cron job:

```
*/5 * * * * curl -s https://example.com > /dev/null || echo "Website Down" | mail -s "Alert"
admin@example.com
```

Purpose:

- Detect downtime quickly
-

10. Clearing Application Cache

Applications store cache files.

Scenario

Clear cache every hour.

Cron job:

```
0 * * * * rm -rf /var/www/cache/*
```

Purpose:

- Improve performance
 - Avoid disk usage growth
-